

Question 1: Water Jug Problem

Problem Formulation:

- Let the state be represented as (x,y) , where x is the water in the 4-gallon jug ($0 \leq x \leq 4$) and y is the water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial State:** $(0,0)$
- **Goal State:** $(2,y)$ where y can be any volume.

Production Rules / Allowable Actions:

1. **Fill 4-gallon jug:** $(x,y) \rightarrow (4,y)$ if $x < 4$
2. **Fill 3-gallon jug:** $(x,y) \rightarrow (x,3)$ if $y < 3$
3. **Empty 4-gallon jug:** $(x,y) \rightarrow (0,y)$ if $x > 0$
4. **Empty 3-gallon jug:** $(x,y) \rightarrow (x,0)$ if $y > 0$
5. **Pour from 3 to 4 until 4 is full:** $(x,y) \rightarrow (4,y-(4-x))$ if $x+y \geq 4$ and $y > 0$
6. **Pour from 4 to 3 until 3 is full:** $(x,y) \rightarrow (x-(3-y),3)$ if $x+y \geq 3$ and $x > 0$
7. **Pour all water from 3 to 4:** $(x,y) \rightarrow (x+y,0)$ if $x+y \leq 4$ and $y > 0$
8. **Pour all water from 4 to 3:** $(x,y) \rightarrow (0,x+y)$ if $x+y \leq 3$ and $x > 0$

Step-by-Step Solution Path:

Step	State (x,y)	Action Applied	Description
1	$(0,0)$	Initial State	Both jugs are empty.
2	$(0,3)$	Rule 2	Fill the 3-gallon jug completely.
3	$(3,0)$	Rule 7	Pour all water from the 3-gallon jug into the 4-gallon jug.
4	$(3,3)$	Rule 2	Fill the 3-gallon jug completely again.
5	$(4,2)$	Rule 5	Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full (takes 1 gallon, leaving 2 gallons in the 3-gallon jug).
6	$(0,2)$	Rule 3	Empty the 4-gallon jug completely onto the ground.
7	$(2,0)$	Rule 7	Pour the remaining 2 gallons from the 3-gallon jug into the empty 4-gallon jug. (Goal Reached)

Question 2: Mars Rover PEAS & Architecture Analysis

1. Percepts

- **Visual Data:** High-resolution digital imagery, stereoscopic depth maps, and infrared camera feeds.
- **Environmental Data:** Ambient temperature readings, atmospheric pressure, wind velocity, and radiation levels.
- **Sensory/Navigation Data:** LiDAR range reflections, inertial measurement unit (IMU) telemetry, wheel slippage sensors, and proximity touch alerts.
- **Chemical/Analytical Data:** X-ray spectrometer returns, microscopic sample images, and gas chromatograph signatures from processed Martian soil/rock.

2. Operating Environment Characterization

- **Partially Observable:** The rover cannot view the entirety of the Martian terrain or subsurface compositions concurrently; it relies on local sensor range.
- **Stochastic:** Environmental fluctuations (e.g., sudden dust storms, mechanical slippage on loose sand dunes) cannot be predicted with total certainty.
- **Sequential:** Current driving decisions and mechanical drilling actions directly influence future placements, energy constraints, and safety profiles.
- **Dynamic:** The physical landscape remains relatively static, but factors like shifting sunlight levels (affecting solar panels) and unexpected atmospheric patterns change dynamically over time.
- **Continuous:** States such as orientation, velocity, distance variables, and continuous sensor parameters occupy infinite, non-discrete spectrums.
- **Single-agent:** The platform coordinates its navigation and laboratory analytics independently without competing against external automated agents.

3. Actions

- **Locomotion:** Moving forward/backward, steering individual wheels, adjusting chassis clearance, and braking.
- **Sample Acquisition:** Extending the robotic arm, positioning the drill assembly, sweeping surface dust, core drilling, and scooping loose soil.
- **Sample Analysis:** Depositing samples into internal baking ovens, executing laser spectroscopy, and conducting microscopic scans.
- **Communication:** Adjusting high-gain directional antennas to transmit telemetry packets back to Earth or orbiting satellites.

4. Performance Evaluation Criteria

- **Scientific Value Generated:** Quantity and novelty of unique rock/soil profiles successfully characterized and transmitted.

- **Safety & Resource Preservation:** Total distance traveled without getting stuck or damaging components, while keeping battery reserves within operational margins.
- **Operational Efficiency:** Minimization of energy consumed per meter traversed or sample refined.

5. Most Suitable Agent Architecture & Justification

The most appropriate framework is a **Goal-Based or Utility-Based Hybrid Agent architecture**.

- **Justification:** A pure reflex framework is insufficient because it lacks the capacity to safely handle complex multi-step objectives under extreme signal latencies (often up to 20 minutes between Earth and Mars).
- A utility-based framework enables the rover to evaluate multiple operational routes or scientific targets concurrently, prioritizing high-yield geological items while balancing energy utilization constraints and thermal survival demands.

Question 3: 8-Queens Problem Formulation

1. Problem Components

- **Initial State:** An empty 8×8 standard chessboard (0 queens placed).
- **Successor Function (Actions):** Place a single queen into the lowest unoccupied column index such that it occupies a row position where it is not immediately attacked by any previously deployed queens.
- **Goal Test:** 8 queens are arranged on the board simultaneously such that no two queens share a matching row, column, or diagonal line.
- **Path Cost:** 0 (we only care about the final valid layout, not the sequence of steps taken to get there).

2. Search Space Exploration Strategies

Incremental Formulation (Tree Search)

- **Approach:** Start with an empty board. At step 1, place a queen in Column 1. At step 2, place a queen in Column 2, and so on.
- **Branching Factor:** If utilizing basic validation rules (no two queens in the same column), the first queen has 8 choices, the second has 8 choices, yielding an upper bound search state space size of $8^8=16,777,216$ states.
- If we restrict placements to prevent rows and columns from overlapping entirely, the search domain contracts down to $8!=40,320$ distinct permutations.

Complete-State Formulation (Local Search / Heuristics)

- **Approach:** Start with all 8 queens already placed on the board (one per column) and iteratively move them to reduce conflicts.
- **Heuristic Evaluation Function (h):** The number of mutually attacking pairs of queens present on the board. For a valid solution state, $h=0$.
- **Applicable Strategy: Hill-Climbing Search or Simulated Annealing.** The agent calculates the h-value for all immediate neighbor configurations (moving a queen within its column) and selects the move that minimizes the number of attacks until $h=0$ is discovered.

Question 4: OLA Cab Booking Problem-Solving Agent

1. Agent Identification

The system uses a **Goal-Based, Problem-Solving Agent** operating under a **Constraint-Satisfaction / Path-Finding** model. The primary goal is to transport the passenger from their origin coordinates to their destination coordinates securely while adhering to user-specified comfort parameters (Mini, Micro, Sedan, Shared, Prime).

2. Pseudocode

Python

function SOLVE-CAB-ROUTING(origin, destination, user_preference) returns action_sequence

 inputs: origin (starting coordinates)

 destination (ending coordinates)

 user_preference (selected ride tier: mini, micro, sedan, prime, shared)

 // 1. Initialize Problem Components

 problem.INITIAL_STATE <- origin

 problem.GOAL_STATE <- destination

 // 2. Filter available drivers based on user preferences

 available_drivers <- GET_ACTIVE_DRIVERS_NEARBY(origin)

 eligible_drivers <- FILTER_BY_TIER(available_drivers, user_preference)

 if eligible_drivers is empty then

 return failure ("No available cabs match your selected ride type.")

 // 3. Select optimal driver based on proximity/ETA

 selected_driver <- FIND_MINIMUM_ETA(eligible_drivers, origin)

 // 4. Formulate the path optimization problem for the transit route

 road_network_graph <- LOAD_LIVE_TRAFFIC_MAP()

 // 5. Search for the optimal path using a search algorithm (e.g., A* or Dijkstra's)

```
action_sequence <- SEARCH_OPTIMAL_PATH(road_network_graph, origin, destination)
```

```
if action_sequence is null then  
  return failure ("Route to destination is unreachable.")
```

```
// 6. Execute actions  
DISPATCH_CAB(selected_driver, origin)  
return action_sequence
```

Question 5: Uniform Cost Search (UCS) Implementation

Uniform Cost Search explores nodes in order of increasing path cost from the start node using a priority queue sorted by cumulative cost $g(n)$.

Step-by-Step Search Trace:

- **Initialization:**
 - Priority Queue (PQ): $[(\emptyset, S)]$
 - Visited/Explored Set: $\{\}$
- **Step 1:**
 - Pop **S** (Cost: 0). Add **S** to Visited.
 - Expand **S**'s neighbors:
 - Path $S \rightarrow A$, Cost = 1. Add to PQ: $(1, A)$
 - Path $S \rightarrow G$, Cost = 12. Add to PQ: $(12, G)$
 - Current PQ: $[(1, A), (12, G)]$
- **Step 2:**
 - Pop **A** (Cost: 1). Add **A** to Visited.
 - Expand **A**'s neighbors:
 - Path $S \rightarrow A \rightarrow B$, Cost = $1+3=4$. Add to PQ: $(4, B)$
 - Path $S \rightarrow A \rightarrow C$, Cost = $1+1=2$. Add to PQ: $(2, C)$
 - Current PQ (Sorted): $[(2, C), (4, B), (12, G)]$
- **Step 3:**
 - Pop **C** (Cost: 2). Add **C** to Visited.
 - Expand **C**'s neighbors:
 - Path $S \rightarrow A \rightarrow C \rightarrow D$, Cost = $2+1=3$. Add to PQ: $(3, D)$
 - Path $S \rightarrow A \rightarrow C \rightarrow G$, Cost = $2+2=4$. Add to PQ: $(4, G)$
 - Current PQ (Sorted): $[(3, D), (4, B), (4, G), (12, G)]$
- **Step 4:**
 - Pop **D** (Cost: 3). Add **D** to Visited.
 - Expand **D**'s neighbors:
 - Path $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$, Cost = $3+3=6$. Add to PQ: $(6, G)$

- Current PQ (Sorted): [(4, B), (4, G), (6, G), (12, G)]
- **Step 5:**
 - Pop **B** (Cost: 4). Add **B** to Visited.
 - Expand **B**'s neighbors:
 - Path $S \rightarrow A \rightarrow B \rightarrow D$, Cost = $4+3=7$. (**D** is already visited/higher cost, ignore).
 - Current PQ (Sorted): [(4, G), (6, G), (12, G)]
- **Step 6:**
 - Pop **G** (Cost: 4). **Goal Node Reached!** Stop execution.

Final Results Summary Table

Path Iteration Options	Explored Node Path Sequence	Total Cumulative Cost
Direct Route	$S \rightarrow G$	12
Via B Branch	$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1+3+3+3=10$
Via C-D Branch	$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1+1+1+3=6$
Optimal Route Found	$S \rightarrow A \rightarrow C \rightarrow G$	$1+1+2=4$

- **Least-Cost Path: $S \rightarrow A \rightarrow C \rightarrow G$**
- **Minimum Transportation Cost: 4 units**

